CS301
Data Structure and Algorithms
Lecture 16: Heap (priority queue)

Pandav Patel
Assistant Professor

Computer Engineering Department
Dharmsinh Desai University
Nadiad, Gujarat, India

Objective

- To understand priority queue
- To understand data structure commonly used to implement priority queue (heap)
- To understand time complexities of operations on heap

heap < max / min

Overview

What is priority queue?

- Queue is FIFO data structure
- Priority queue is a queue where all the elements have priority along with value
- Element with the higher priority will leave the queue before other elements with lower priority than that element, irrespective of the sequence in which they were inserted in the queue
  - It can be other way around too
- For this lecture, we will assume that the elements with the same priority can leave the queue in any order
  - Once you understand overall approach this can be easily addressed
  - This link explains how we can preserve order of elements with same priority using FIFO rule

- How will you implement such a queue?

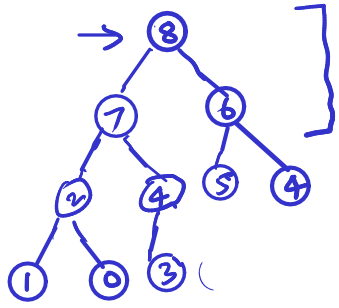
Priority queue: example and applications

- Three separate lines (with different amount) in famous temples (e.g. 100rs, 200rs, 500rs)
  - What data structure would you use to code something similar? What will be the complexity of enqueue and dequeue?
- What if there are many lines with different amount? Whoever gives more money, gets ahead in queue than everyone else who paid less than him/her
  - What data structure would you use to code something similar? What will be the complexity of enqueue and dequeue?

- Applications
  - Process scheduling
  - To implement Prim's algo and Dijkstra's algo
  - Heap sort
  - etc...

How to implement a priority queue?

- Properties of heap
  - (1) Structural property
  - (2) Heap property
    - parent ≥ child → Max heap
    - parent ≤ child → Min heap



Max heap

- $O(\log(n))$ → Insert an element
- $O(\log(n))$ → Delete max/min element
- → get max/min → $O(1)$

$n - \lceil \log(n) \rceil \geq$ height of tree

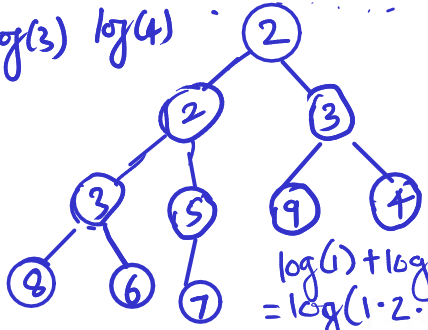
Find

n random numbers — min/max heap

① Time complexity of creating min/max heap from (n) random numbers?

7 2 9 8 3 4 3 2 6 5

log(1) log(2) log(3) log(4) . . . . . log(n)



min heap

parent ≤ child

THINK!

Can you do this in O(n)?

$\log(1)+\log(2)+\ldots+\log(n)$
$=\log(1\cdot 2\cdot 3\cdot \ldots \cdot n)$
$<\log(n\cdot n\cdot n\cdot \ldots \cdot n)$
$=\log(n^n)=n\log(n)$

hence $O(n\log(n))$

Can you do sorting using heap? of n random numbers

space complexity of heap sort?

→ | 2 | 7 | 8 | 9 | 10 | 11 | 12 |

To create heap
— $n \log(n)$

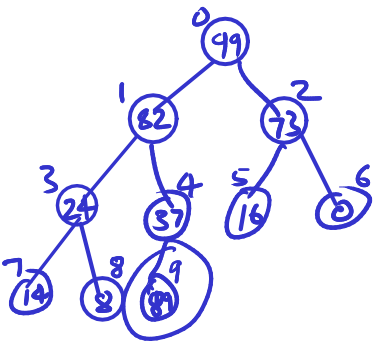
To delete n Element
$n \times \log(n)$

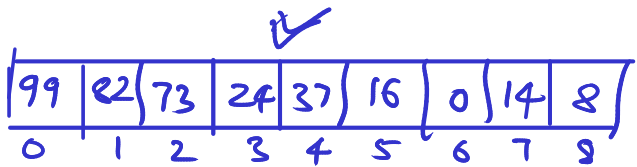
Heap Sort

$O(n\log(n) + n\log(n)) = O(n\log(n))$

How to create heap data structure in memory?

Using array/vector



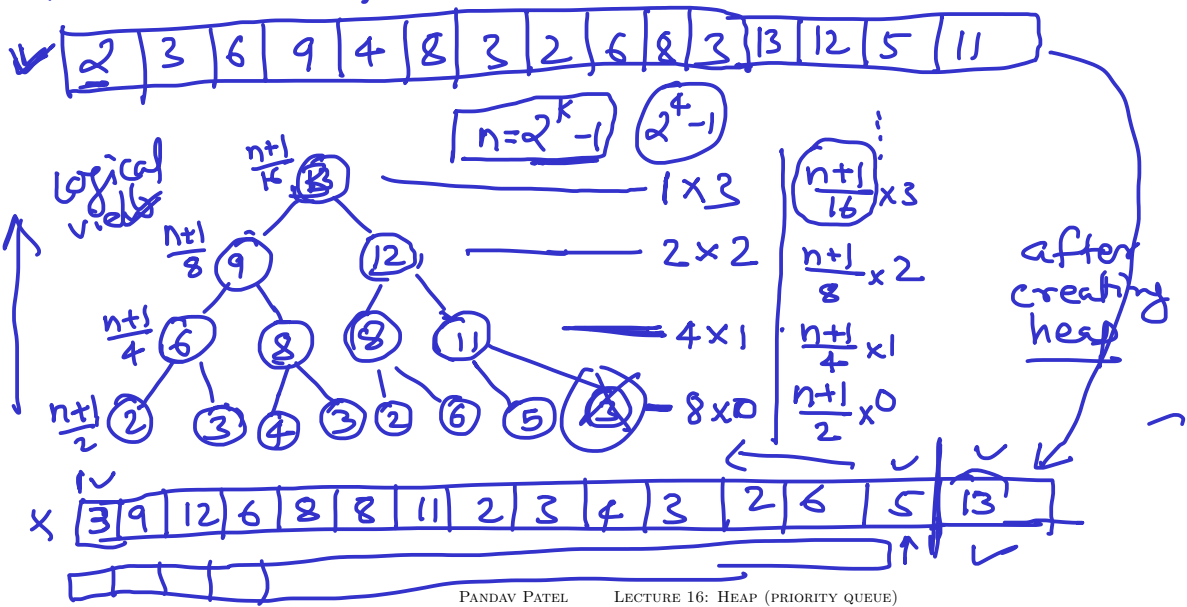


index of left child : parent index $\times 2 + 1$

" " right " : " " $\times 2 + 2$

parent index $= (\text{child index} - 1)/2$

How to efficiently create a heap from n random numbers?



Total swaps $= \frac{n+1}{4} \times 1 + \frac{n+1}{8} \times 2 + \frac{n+1}{16} \times 3 + \cdots + \frac{n+1}{2^k} \cdot (k-1)$

$$S = (n+1)\left(\frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \cdots + \frac{k-2}{2^{k-1}} + \frac{k-1}{2^k}\right) \quad (1)$$

$$2S = (n+1)\left(\frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \cdots + \frac{k-1}{2^{k-1}}\right) \quad (2)$$

(2) − (1)

$$2S - S = (n+1)\left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \cdots + \frac{1}{2^{k-1}}\right) \boxed{- (n+1) \times \frac{k-1}{2^k}} \quad \text{ignore}$$

$$S < (n+1) \sum_{i=1}^{k-1} \frac{1}{2^i} \quad (3)$$

$$S < (n+1) \sum_{i=1}^{\infty} \frac{1}{2^i} \quad (4)$$

GP — $a = 1/2, r = 1/2$

$$S < (n+1) \frac{1/2}{1 - 1/2}$$

$$S < (n+1) \quad O(n)$$

How to do heap sort with O(1) space?

→ Can we utilize the same array which stores heap to store sorted array?

Ans = Yes, after deleting an element from heap, it size reduces by 1 and that empty slot at end of array (in which heap is stored) can be used to store deleted element.

Note: Max heap will result in ascending order.